

1. Why is 'record_hash' useful for rerun-safe loading, and which columns should not be included in it?

It is useful because it generates a unique cryptographic fingerprint of a row's core business attributes. This enables deterministic upserts and duplicate detection so that rerunning the pipeline doesn't create duplicate records or redundant database writes. Columns to exclude are volatile metadata columns. If included, the hash value would change on every execution even if the underlying business data remains identical.

2. Why should raw data usually be preserved even when staging/curated outputs are sufficient for analytics?

Raw data acts as an immutable single source of truth. Preserving it ensures that if transformation logic has a bug, business requirements change, or downstream schemas evolve, I can just always safely reprocess and backfill historical data without needing to re-fetch from external systems.

3. What is the difference between a data-quality rejection and a system exception?

Data quality rejection refers to the business or schema failure within the data itself. It reflects on invalid rows, which are typically routed to a quarantine table or dropped, allowing the rest of the valid pipeline to continue running. System exception refers to the infrastructure or runtime failure. It crashes or halts the pipeline because the execution environment or system state is broken.

4. Why might Parquet outperform CSV for selected analytical workloads even if both contain the same rows?

Parquet is a columnar storage format with built-in data compression and embedded metadata statistics. This allows analytical query engines to use predictive pushdown and columnar projection, drastically reducing disk I/O compared to parsing row-oriented CSV files line by line.

5. Why is a DAG that contains all transformation logic directly considered harder to maintain?

Embedding heavy transformation logic directly inside a DAG couples orchestration concerns with business logic. It makes unit testing extremely difficult, clutters code readability, prevents local script execution without spinning up Airflow, and risks bloating the Airflow schedulers' parsing time if heavy libraries or computations are evaluated at import time.

6. How do retries interact with idempotency? Give an example where retries without idempotency cause damage.

Idempotency ensures that running an operation multiple times yields the exact same state as running it once. Safe retries depend entirely on idempotency; if a task fails midway and retries, an idempotent operation prevents duplicate data or corrupted states.

A load task that executes a standard insert statement without an upsert or unique constraint is an example of no idempotency. If the task fails right before Airflow records its completion status, Airflow will retry the task, executing another INSERT and resulting in duplicate rows in the target table.

7. What trade-off is introduced by partitioning too aggressively?

Partitioning too aggressively leads to the small files problem. This creates an explosion of tiny files or table partitions, increasing metadata overhead, slowing down directory listings, and degrading query engine performance due to excessive file-opening overhead.

8. How would you adapt the pipeline if the source became an API or database instead of local files?

Swap out the file-reading logic inside the extraction module for an API client or database connection query. Instead of reading local files from disk, the extractor would pull data incrementally and save the payloads into the standard raw storage zone. This keeps all downstream staging, transformation, and loading modules completely isolated from the upstream source change.